

TEL102 - Práctico 3

Semestre 2, 2025

Fecha: 27/10/2025, Duración: 1h 10min (45 min Partes A y B, 25 min Parte C)

Instrucciones generales

- **Parte A: análisis individual en papel.** El estudiante debe examinar un programa C++ orientado a objetos y determinar su salida exacta, justificando brevemente el orden de ejecución y el comportamiento de los métodos virtuales.
- **Parte B: diseño individual en papel.** A partir del mismo programa, cada estudiante debe **diseñar una nueva clase** que amplíe o complemente su funcionalidad. Se evaluará la coherencia del diseño, la correcta definición de atributos, métodos, relaciones (herencia o composición) y la claridad del propósito de la clase.
- **Parte C: implementación asistida por IA.** Utilizando el diseño realizado en la Parte B, el estudiante debe **implementar su clase en código C++**, integrándola al programa base. Puede usar **modelos de IA** como apoyo, siempre que documente su uso en un archivo **Readme.md** que incluya:
 - **Información del estudiante:** nombre completo y rol.
 - **Prompts utilizados** con el modelo de IA.
 - **Información del modelo** (nombre, versión, empresa, etc.)
 - **Modificaciones o correcciones** hechas al código sugerido por la IA.
 - **Reflexión breve** sobre cómo la IA apoyó su aprendizaje en este ejercicio.

Tiempo total: 1 hora y 10 minutos

- **Partes A y B:** 45 minutos en conjunto
- **Parte C:** 25 minutos en computador

Los archivos base (**main.cpp**, **figuras.h**, **figuras.cpp**) estarán disponibles en **AULA USM** y en el enunciado. El código inicial contiene una jerarquía de clases (**Figura**, **Rectangulo**, **Circulo**) que el estudiante deberá analizar. La nueva clase diseñada debe integrarse a esta jerarquía.

Compilación sugerida: `g++ -Wall -Wextra main.cpp figuras.cpp -o practico3`

Enunciado: Sistema de Figuras: Herencia y Polimorfismo

Se entrega un programa C++ que implementa una jerarquía simple de figuras geométricas (**Figura**, **Rectangulo**, **Circulo**) y su uso.

Su tarea será:

1. **Analizar** el código entregado.
2. **Diseñar** una nueva clase que amplíe la jerarquía.
3. **Implementar** la clase diseñada en C++, integrándola al código con asistencia de IA.

Parte A: Análisis del código (en papel)

Analice el programa entregado, compuesto por los archivos **main.cpp**, **figuras.cpp** y **figuras.h**. El programa implementa una jerarquía simple de clases para figuras geométricas (**Figura**, **Rectangulo**, **Circulo**), demostrando el uso de **herencia** y **polimorfismo** en C++.

Responda las siguientes preguntas en el mismo orden en que aparecen las secciones marcadas en el código fuente:

A.1: Explique por qué se puede llamar a los métodos **nombre()** y **area()** de la clase **Figura** en la función **imprimir**, a pesar de haber sido definidos como virtuales puros.

A.2: Explique por qué es necesario pasar las variables de tipo **Rectangulo** y **Circulo** con el operador de referencia (&) tanto en **push_back()** como en la función **imprimir()**.

A.3: Escriba la salida esperada por consola al ejecutar el programa completo.

Nota: No es necesario copiar el código; basta con escribir la salida textual que se imprimiría en la consola, incluyendo los mensajes de creación de figuras y las líneas de área mostradas.

Parte B: Diseño de nueva clase (en papel)

Diseña una nueva clase llamada **Triangulo** que amplíe el programa entregado y sea **compatible con los mismos tipos de llamados** que se realizan en el código actual, por ejemplo:

```
Triangulo t1(3.0, 4.0);
imprimir(&t1);
```

En este ejemplo, **3.0** representa la **base** y **4.0** la **altura** del triángulo.

Entregue en papel: la **definición de la clase Triangulo**. No es necesario implementar los métodos.

Parte C: Implementación asistida por IA (en computador)

Implemente la clase **Triangulo** en el proyecto, **basándose en el diseño realizado en papel** (Parte B). Si la implementación no se ajusta a dicho diseño, **no será considerada en la evaluación**.

Instrucciones

- Solo se puede crear la clase **Triangulo**.
- La **definición** de la clase debe estar en **figuras.h**.
- Su **implementación** debe estar en **figuras.cpp**.
- No se debe modificar el resto del código
- Una vez implementada la clase, **descomente las líneas indicadas en el archivo main.cpp** para integrar el nuevo objeto:

```
/* Parte C: Descomentar las siguientes líneas solo en la Parte C */
Triangulo t1(3.0, 4.0);
figs.push_back(&t1);
```

De esta forma, el programa mostrará también la información del triángulo junto con las demás figuras.

Entrega

Archivo: TEL102_P3_Nombre_Apellido.zip **Contenido:**

- figuras.h** (con la definición de **Triangulo**)
- figuras.cpp** (con la implementación de **Triangulo**)
- Readme.md**

Puntaje (10 pts)

Parte A (3 pts)

- Explicaciones A.1 y A.2 correctas: 2 pts
- Salida completa y en orden: 1 pt

Parte B (3 pts)

- Definición coherente de la clase **Triangulo** (herencia, atributos, métodos): 2 pts
- Compatibilidad con los llamados del programa: 1 pt

Parte C (4 pts)

- Implementación funcional en C++ coherente con el diseño: 2 pts
- Integración y compilación correctas: 1 pt
- Documentación y uso de IA en **Readme.md**: 1 pt

Códigos

figuras.h

```
#include <string>

class Figura {
public:
    virtual std::string nombre() = 0;
    virtual double area() = 0;
};

class Rectangulo : public Figura {
private:
    double a_, b_;
public:
    Rectangulo(double a, double b);
    std::string nombre();
    double area();
};

class Circulo : public Figura {
private:
    double r_;
public:
    Circulo(double r);
    std::string nombre();
    double area();
};
```

figuras.cpp

```
#include "figuras.h"
#include <iostream>
#include <cmath>

Rectangulo::Rectangulo(double a, double b) : a_(a), b_(b) {
    std::cout << "(+) Rectangulo(" << a_ << ", " << b_ << ")\n";
}

std::string Rectangulo::nombre() { return "Rectangulo"; }

double Rectangulo::area() { return a_ * b_; }

Circulo::Circulo(double r) : r_(r) {
    std::cout << "(+) Circulo(" << r_ << ")\n";
}

std::string Circulo::nombre() { return "Circulo"; }

double Circulo::area() { return 3.141592653589793 * r_ * r_; }
```

main.cpp

```
#include <iostream>
#include <vector>
#include "figuras.h"

/* A.1 Explique por qué se puede llamar a los métodos nombre() y area()
   de la clase Figura en la función imprimir, a pesar de ser definidos
   como virtuales puros*/
void imprimir(Figura* f) {
    std::cout << f->nombre() << " -> area=" << f->area() << "\n";
}

Figura* seleccionar(std::vector<Figura*> v, int i) {
    return v.at(i);
}

int main() {
    std::cout << "[Inicio]" << std::endl;

    Rectangulo r1(1.0, 2.0);
    Rectangulo r2(2.0, 3.0);
    Circulo c1(2.0);

    std::vector<Figura*> figs;
    /* A.2 Explique por qué es necesario pasar las variables de tipo Rectangulo
       y Circulo con el operador de referencia (&) tanto en push_back() como
       en la función imprimir()
       */
    figs.push_back(&r2);
    figs.push_back(&c1);

    /* Parte C: Descomentar las siguientes líneas solo en la Parte C*/
    // Triangulo t1(3.0, 4.0);
    // figs.push_back(&t1);

    std::cout << "[Uso de objeto local]\n";
    imprimir(&r1);

    std::cout << "[Recorrido de vector]\n";
    for (int i = 0; i < figs.size(); i++) {
        imprimir(figs[i]);
    }

    std::cout << "[Fin main]\n";
    return 0;
}
```